

The Task for Composite Experiments

on Database System

1. Objective:

The objective of this course is learning to solve the practical problem using the knowledge on database system. From this process, the student can exercise how to design the database for a practical case, how to create the database including tables, views, triggers, primary keys etc, how to use SQL commands to complete querying tasks and other tasks, and how to use a programming language to communicate with the database. This project aims to provide a realistic experience in the conceptual design, logical design, implementation, operation, and maintenance of a small relational database.

The project can go well beyond the minimal requirements I outline at the end. I encourage such extensions. They could turn into a senior design project or other independent work.

The description given here of the enterprise you are modeling is necessarily somewhat vague and incomplete. You will need to fill in the holes in this document in create a precise design and concrete implementation of the interfaces and applications using the database.

2. Project Description

The **Online Store Database Project** is designed to streamline and manage core e-commerce operations efficiently. This database system focuses on maintaining key entities like products, customers, orders, payments, and inventory, ensuring seamless business operations for an online store. The system enables the tracking of product information, customer details, order processing, payment tracking, and inventory management. By implementing this project, businesses can handle complex tasks such as adding new products, managing customer orders, processing payments, and monitoring inventory levels effectively.

✓ Understanding Project Requirements:

- Identify the key aspects of an online store such as product catalog, customers, orders, payments, and inventory.
- Analyze the business needs according to the following roles:
 - **Customer** – Browses products, adds items to cart, places orders, makes payments, views order history, and uses membership points.
 - **Manager** – Manages product catalog, processes orders, monitors sales reports, handles customer accounts, and controls system settings.
 - **Cashier** – Quickly finds products, processes checkout, accepts

payments, applies member discounts, and prints receipts for in-store or phone orders.

- **Inventory Clerk** – Tracks stock levels, updates inventory after deliveries or sales, sets reorder thresholds, and generates low-stock alerts.
- **Supplier** – Provides product information, receives purchase orders, updates product availability and prices, and manages supply contracts.
- Focus on functionality like product management, customer details, order tracking, payment processing, and inventory updates.

✓ **Identify relationships:**

- Orders placed by customers should be linked to products and payments.
- Inventory should track product stock levels.
- Customers, orders, and payments must be interrelated with foreign keys.
- Plan constraints: Ensure data integrity by defining primary keys, unique constraints, and foreign key references between tables.

✓ **Database Design about tables, views, triggers:**

- Create **at least 5 tables** such as Products, Customers, Orders, Payments, and Inventory with appropriate fields, data types, primary keys, and constraints.
- Follow a similar structure for the other tables (Customers, Orders, Payments, Inventory), ensuring foreign keys are used to maintain relationships between tables.

- Create **at least 3 views**. Examples are provided as follows:

- **Cashier-Facing View — Product Sales Information View**

Description: Provides cashiers with quick access to product information during checkout, displaying only basic information of available products.

Requirements: Only display products with an "Available" status; hide sensitive information such as purchase price and supplier details.

- **Management-Facing View — Product Sales Statistics View**

Description: Allows management to view sales summaries for each product to support sales analysis and decision-making.

Requirements: Aggregate sales data from completed orders; sort results by total sales amount in descending order.

- **Member-Facing View — Member Discount Products View**

Description: Allows members to view products currently eligible for member discounts or special offers, enhancing the member shopping experience.

Requirements:

Only display products with an "Available" status that have member discounts

Only show products with stock quantity greater than zero

Hide sensitive information such as purchase price and supplier details

- Create **at least 2 triggers**. Examples are provided as follows:

- **Low Stock Warning:** When an order is inserted or updated (i.e., products are

purchased), inventory is automatically deducted. If the remaining stock of any product falls below its predefined threshold, the system generates a warning record to notify the administrator

- **Automatically Calculate Member Points:** When an order status is updated to "completed", the system automatically calculates the member points based on the order total amount. The points are calculated using the formula: $\text{points} = \text{floor}(\text{total_amount}) \times \text{points_rate}$ (default 1 point per 1 unit of currency). The computed points are then added to the member's total_points balance, and a corresponding record is inserted into the points_records table. The trigger must ensure that each order is processed only once to avoid duplicate points.
- **At a minimum, you should create at least 4 indexes to improve query performance.** You must write the corresponding CREATE INDEX statements and explain your choices in the report.

✓ **Insert Sample Data:**

- Add some sample data for testing purposes. Insert records into the tables using INSERT statements.

✓ **Establish Relationships Between Tables:**

- Ensure at least 3 types of relationships between tables are correctly established using foreign keys:
 - Orders are linked to Customers.
 - Payments are linked to Orders.
 - Inventory is linked to Products.

✓ **Develop CRUD Operations:**

- **Create:** Insert new products, customers, and orders into the system.
- **Read:** Retrieve product information, customer profiles, order statuses, and inventory levels.
- **Update:** Update product details, customer information, or order statuses (e.g., marking orders as "Completed").
- **Delete:** Remove unnecessary or obsolete products, orders, or customer records.

3. Requirement

- 1) **Database:** Microsoft's SQL Server or MySQL are both fine. You can select one to complete your experiment.
- 2) **Working:** You need to create all tables, attributes, domains, primary and foreign keys, constraints, views and indexes with SQL DDL sentences, do querying with

SQL DML sentences.

- 3) **If possible, please use one advanced programming language (C/C++, Java, C#, etc) to develop a GUI program to realize all functions and querying above-mentioned. And you will get extra marks and a higher-level evaluation.**

- 4) **Experiment Report:** Your experiment report must include several parts as follows:

- **Part I: Cover Sheet:** One cover sheet including the title of the report, your name and student ID, and submit date, etc;
- **Part II: Project Introduction and Requirement:** Please provide a comprehensive description of the project, and specify the functions designed for each user role.

- **Part III: Project Design:**

Construct an E-R diagram representing the conceptual design of the database. At minimum you must include all the entity and relationship sets implied by this handout. You may go beyond the minimum. In your E-R diagram and its explanation, you MUST:

- Clearly label mapping cardinalities (e.g., 1:1, 1:N, M:N) for each relationship.
- Please specify or draw whether the entities at both sides for the relationship are total or partial participation.
- List all entities, relationships, their attributes, and specify which attributes form the primary key for each entity.
- For the relational schema derived from the **E-R diagram**, state which normal form (e.g., 3NF, BCNF) it satisfies and justify your answer.

Each E-R model must be accompanied by an explanation in your own words.

- **Part IV: Relational Model**

Convert the E-R schema to a relational database schema. Be sure to include appropriate indices and constraints. Be sure your relational schema conforms to your E-R schema. Write down all SQL sentences (DDL, DML, views, triggers, indexes) with an explanation. However, provide snapshot pictures

(screenshots) only for the following critical SQL statements:

- CREATE TABLE statements (at least two representative tables)
- CREATE VIEW statements (all views)
- CREATE TRIGGER statements (all triggers)
- Include at least 10 single-table queries and 5 complex multi-table select query statements. (e.g., involving JOIN, GROUP BY, or subqueries)
- For other statements (e.g., simple INSERT, basic UPDATE), you may include the SQL code with sample data by screenshots.

Make sure each screenshot is clearly readable and shows the execution environment.

- **Part V: Project Conclusion:** Make a summary of your working with at least 300 words.

- **System Demonstration:**

- **(optional)** Demonstration of system functions (screenshots or live demo)
- **Insert sample test data and test all queries, tables, views, triggers (screenshots or live demo)**

5) **System demonstration and presentation. Please come to demonstrate and defend your work at the scheduled appointment time.**

6) **Submission: Document (word or pdf) + database backup files (generated SQL scripts) + source codes (optional) – if provided, extra credit will be given.**

7) Where to submit: 学习通 APP.

8) **Deadline: July 10th, 2026.**

9) **Offline database system demo from 9:00 a.m. to 3:00 p.m. on 7th July and 9th July**

10) **Note: All students need submit their database backup, it must include all tables (contained attributes, primary keys, foreign keys), views, triggers, indexes, and other SQL objects, excluding the actual data records. All these objects (except data records) must have the last 3 digits of your student ID appended as a suffix to distinguish them from those of other students.**

11) **If developed applications are provided, please include running screenshots of**

each function in the report.